

# System Design Specification: Zero-Shot Pattern Detection

|                      |                                                                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project</b>       | DINO Pattern Detector                                                                                                                             |
| <b>Task</b>          | Zero-shot pattern detection in black-and-white technical BOM drawings                                                                             |
| <b>Author</b>        | Võ Tá Quang Nhật                                                                                                                                  |
| <b>GitHub</b>        | <a href="https://github.com/votaquangnhat/sotatek-assessment">https://github.com/votaquangnhat/sotatek-assessment</a>                             |
| <b>Demo</b>          | <a href="https://huggingface.co/spaces/votaquangnhat/sotatek-assessment-hf">https://huggingface.co/spaces/votaquangnhat/sotatek-assessment-hf</a> |
| <b>Main approach</b> | Frozen DINOv2 ViT-S/14 features + cosine-similarity heatmap + multi-scale search + NMS                                                            |

## 1. Problem Analysis and Approach

The task is to detect every occurrence of a provided pattern inside a black-and-white technical BOM drawing. The system must work in a zero-shot setting: changing the input pattern should not require retraining, fine-tuning, or adding a new class label.

This problem is difficult because the drawings are often high-resolution, sparse, and line-based. Many symbols are small, visually similar, and may appear at slightly different scales or rotations.

I attempted to solve this problem using deep feature similarity. I chose this approach because it gives a balance between zero-shot ability and implementation time. My initial thought was that classical template matching is simple, but it is sensitive to scale changes, small shifts, and line thickness differences. Training a detector or a Siamese network would require labeled data, which does not fit the zero-shot requirement. Foundation-model-based feature matching is not perfect for technical drawings, but it allows the system to compare a new query pattern with the drawing without retraining.

I use a frozen DINOv2 ViT-S/14 backbone as a general-purpose feature extractor. Both the pattern and the drawing are converted into patch-level DINO features. The pattern is resized so that its grid size is approximately  $5 \times 5$  tokens. I also estimate an ROI for the drawing image to reduce the search area. The pattern features are then slid over the ROI features to produce a cosine-similarity heatmap. Peaks in the heatmap become candidate detections, and Non-Maximum Suppression (NMS) is used to remove duplicate boxes.

This keeps the system zero-shot because the model is never trained on the target patterns. The pattern itself acts as the query at inference time.

## 2. System Pipeline

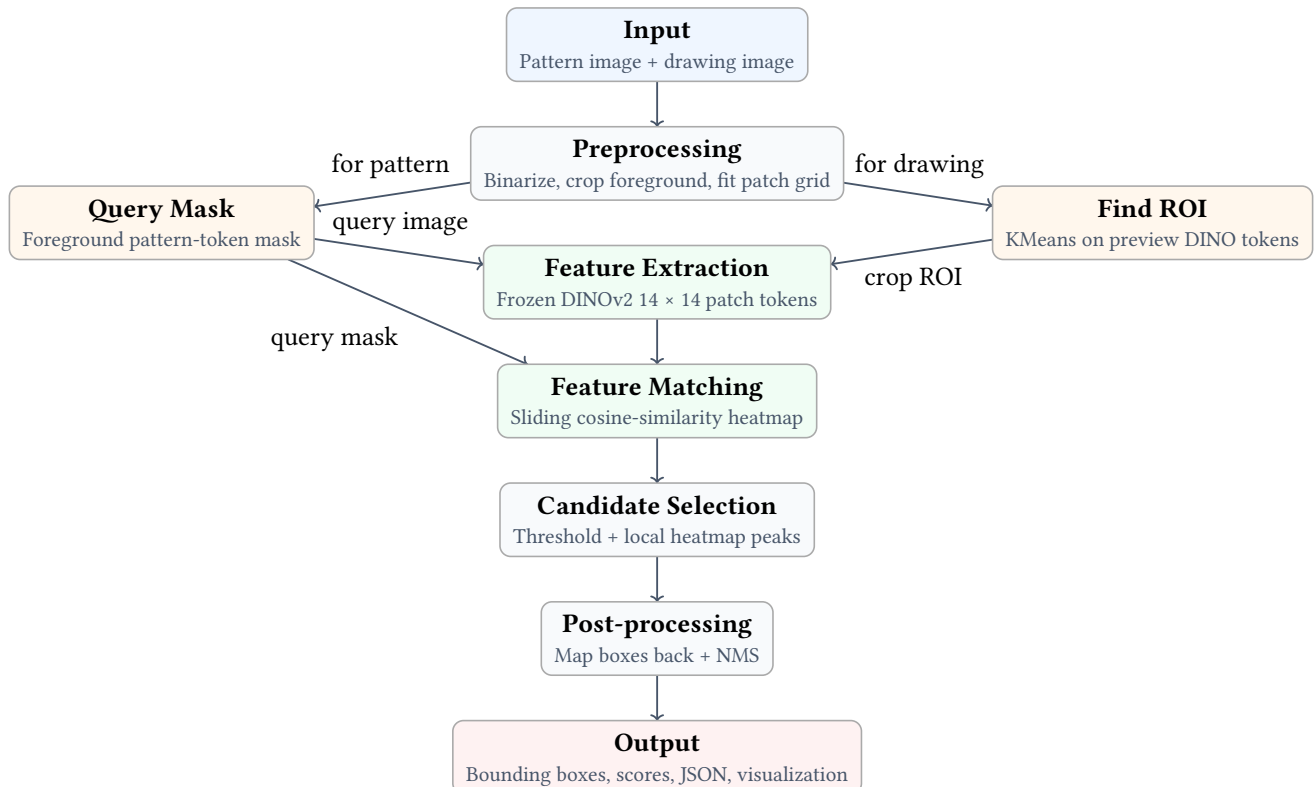


Figure 1: System pipeline of the proposed zero-shot pattern detector.

The inference flow is:

1. Receive one cropped pattern image and one full drawing image.
2. Convert both images to binary form and align them to the  $14 \times 14$  DINO patch grid.
3. Optionally estimate a Region of Interest (ROI) in the drawing to reduce the search area.
4. Extract frozen DINOv2 patch tokens for the pattern and the drawing.
5. Compute a sliding cosine-similarity heatmap in feature space.
6. Select local heatmap peaks using a percentile threshold or an absolute score threshold.
7. Convert token locations back to pixel bounding boxes.
8. Apply NMS, sort the detections by score, and return the visualization and JSON output.

### 3. Module Details

#### 3.1 Image preprocessing

All input images are first converted to grayscale and binarized using a fixed threshold. Black pixels represent foreground drawing strokes, while white pixels represent the background. The foreground area is then cropped to remove unnecessary white margins.

Each image is adjusted so that its width and height are divisible by the DINO patch size. This is necessary because DINOv2 ViT-S/14 produces one token for each  $14 \times 14$  image patch. For drawing images, the system crops the image to the nearest valid patch grid.

For pattern images, the pattern is resized so that its longer grid side has length 5. After that, the shorter side is padded, if needed, so that the final pattern grid becomes  $5 \times 5$ . I chose this size as a trade-off between effectiveness and efficiency. My intention was to make non-square patterns easier to compare during heatmap computation. However, this idea did not work as well as expected, and I did not have enough time to fix it completely.

This preprocessing step produces stable token grids and makes the sliding feature comparison easier.

##### 3.1.1 Find ROI for drawing image

For large drawings, running feature matching over the full image is expensive and may introduce many irrelevant candidates. Therefore, the system includes an optional automatic ROI proposal step.

The drawing is first resized to a small preview scale. DINO features are extracted from this preview image, and KMeans clustering is then applied directly to the token features. I chose 3 clusters based on the assumption that the drawing can roughly be separated into three types of regions: the main drawing region, the text region, and the empty background region.

After clustering, the cluster label at the center token is selected as the likely main drawing region. Small connected components are removed as outliers, and the bounding box of the remaining tokens is converted back to pixel coordinates.

If ROI estimation fails or produces invalid coordinates, the system falls back to using the full drawing.

##### 3.1.2 Find mask of pattern image

The pattern foreground mask is computed at token resolution. The processed binary pattern is resized to its token grid, binarized again, and black foreground locations are marked as valid query tokens.

This mask is useful because the pattern crop often contains white background areas. With the mask, the feature comparison focuses more on the actual shape of the pattern. During matching, only foreground query tokens contribute to the similarity score, which helps reduce background bias.

#### 3.2 Feature Extraction

The feature extractor is a frozen DINOv2 ViT-S/14 model loaded from the official DINOv2 repository. The model is used only in inference mode. No gradient computation, training, or fine-tuning is performed.

The preprocessing transform converts the binary RGB image into a tensor and applies ImageNet normalization. The output patch tokens from DINOv2 are reshaped into a 2D feature grid.

#### 3.3 Feature Matching

The pattern is treated as a query feature window, while the drawing is treated as a larger feature map. Both the pattern and drawing features are L2-normalized, so their dot product becomes cosine similarity.

The query mask is multiplied with the pattern feature window. Then, the system uses a 2D convolution operation to slide the query over the drawing feature grid efficiently. Each heatmap value is the average cosine similarity between the foreground query tokens and the corresponding drawing window.

To support scale variation, the drawing can be evaluated at multiple pre-selected scales. The current default scale values are chosen empirically, but the demo allows the reviewer to edit the scale list. To support simple rotation variation, the system also evaluates a 90-degree rotated version of the pattern.

### 3.4 Post-processing

Candidate boxes are selected from the heatmap in two steps:

1. A score threshold is chosen. This can be either an absolute score threshold or a percentile threshold over the heatmap values. After some experiments, I chose the 90th percentile as the default.
2. Only local maxima are selected. I use max-pooling so that nearby high-score locations do not all become separate candidates.

Each candidate token location is converted back to pixel coordinates. When matching is done on a scaled drawing, the bounding box is mapped back to the original drawing coordinates by dividing by the scale factor. If ROI mode is enabled, the ROI offset is added back.

Finally, NMS removes overlapping boxes based on IoU. The remaining boxes are sorted by confidence score.

The output visualization draws red bounding boxes directly on the original drawing, together with the score, drawing scale, and detection index.

## 4. Demo Interface

The demo is implemented with Gradio and deployed on HuggingFace Spaces. It provides:

- Pattern image upload.
- Drawing image upload.
- Built-in examples for quick testing.
- A tuning panel for ROI, drawing scales, heatmap threshold, absolute score threshold, NMS IoU, maximum number of detections, and debug mode.
- Detection result image with bounding boxes.
- ROI preview image.
- JSON output containing bounding boxes, scores, runtime, model name, device, drawing scales, and ROI status.

The model is cached after the first loading step, so repeated requests do not reload DINOv2 every time.

## 5. Strengths and Weaknesses

### Strengths.

- Fully zero-shot: the query pattern can change without retraining.
- Uses frozen DINOv2 features.
- Supports multiple detections in the same drawing.
- Supports simple rotation search using a rotated query variant.
- Uses foreground query masking to reduce the effect of white background.

### Weaknesses.

- Detection quality depends heavily on the chosen scale list and threshold.
- The  $14 \times 14$  token resolution limits localization precision, especially for very small symbols.
- Fixed binarization may fail on noisy scans, gray backgrounds, or low-contrast drawings.
- DINOv2 features are trained on natural images, not specifically on technical drawings.
- Computing the heatmap takes a long time on the CPU used by HuggingFace Spaces.
- If the pattern shape is too simple, the mask may not work as intended. This becomes worse when the drawing lines are very thin.
- The system cannot reliably detect the exact number of pattern occurrences in the drawing. It mainly returns the candidates with the highest scores.
- The system does not work well when the pattern width-to-height ratio is far from 1.

## 6. Current Limitations and Future Improvements

The current system is a functional zero-shot prototype, but several parts are not fully solved yet.

### 1. Scale search is still manual.

The current system uses pre-selected drawing scales. This works for some examples, but it is not robust enough for all pattern sizes. A better version could use an automatic scale pyramid or estimate the likely scale from the pattern and drawing token statistics.

2. **The system cannot reliably detect the exact number of pattern occurrences.**

It mainly returns the top-scoring candidate boxes. If the threshold is too low, false positives may appear. If the threshold is too high, some true patterns may be missed.

3. **Non-square patterns are difficult.**

The current preprocessing resizes the pattern into a fixed  $5 \times 5$  token grid. This keeps the feature window small, but it does not work well when the pattern width-to-height ratio is far from 1.

4. **Very thin or very simple patterns are difficult.**

Since DINOv2 works at patch level, very thin line structures may not be represented clearly enough in the token features. Simple patterns may also produce weak or ambiguous foreground masks.

5. **The ROI proposal is heuristic.**

The ROI is estimated by KMeans clustering on preview DINO features. It can reduce the search area, but it may fail on drawings with unusual layouts or when the center region does not correspond to the main drawing area.

6. **Runtime on CPU is slow.**

Computing heatmaps for multiple scales can take a long time on CPU, especially on HuggingFace Spaces. Runtime could be improved by caching drawing features, batching multiple scales, and pruning the search area more aggressively.

7. **The model is not trained specifically for technical drawings.**

DINOv2 is a general visual feature extractor, mostly trained on natural images. It is useful for zero-shot matching, but it may not be optimal for sparse, line-based technical drawings.

Given more time, I would improve the system in the following directions:

1. Add automatic scale search using a scale pyramid.
2. Improve the handling of non-square patterns.
3. Add better preprocessing for scanned drawings, such as adaptive thresholding, denoising, and line thickening.
4. Cache drawing features when searching for multiple patterns in the same drawing.
5. Batch multiple scales to improve runtime.
6. Experiment with other feature extractors instead of relying only on DINOv2 ViT-S/14.

I am also not fully sure whether this deep-learning-based approach is the best fit for this specific type of technical drawing problem. It works as a zero-shot prototype, but the results suggest that feature-based matching still has practical issues for very thin, simple, or non-square patterns.

## 7. Conclusion

This project implements a zero-shot pattern detection system for black-and-white technical BOM drawings. The system uses a frozen DINOv2 ViT-S/14 model to extract patch-level features from both the query pattern and the drawing. It then computes a cosine-similarity heatmap between the pattern and the drawing ROI, selects local peaks as candidate detections, and applies NMS to remove duplicate boxes.

The main advantage of this approach is that it does not require training or fine-tuning for each new pattern. The user can provide a new pattern image, and the system can use it directly as a query. However, the current version is still a prototype. Its performance depends on scale selection, threshold choice, pattern shape, and drawing quality. In particular, very thin patterns, very simple patterns, and non-square patterns remain difficult cases.

Overall, the system demonstrates a possible zero-shot deep feature matching pipeline, but it still needs better scale handling, faster inference, and more robust preprocessing before it can be considered reliable for more general technical drawings.

## References

- [1] Maxime Oquab et al. "DINOv2: Learning Robust Visual Features without Supervision." 2023.
- [2] Poulami Sinhamahapatra et al. "Finding Dino: A Plug-and-Play Framework for Zero-Shot Detection of Out-of-Distribution Objects Using Prototypes." 2025.